

Explainability

Explainable AI

Explainable AI (XAI) is a branch of artificial intelligence focused on making AI systems understandable to humans. Instead of acting like a “black box” that gives answers without explanation, explainable AI shows how and why a model made a decision.

XAI helps people:

- Trust AI systems by understanding decisions
- Detect bias or errors in models
- Meet legal and ethical requirements
- Improve transparency and accountability

Feature Attribution Methods

The aim of these methods is to identify which features are most important for the prediction

- In linear models, the coefficients provide a direct estimate of feature importance
- In Decision Trees, the feature importance can be estimated based on its contribution to reduction in impurity

In non-linear models (e.g. ANNs and kernel SVMs) features importance is less directly observable

Common feature attribution methods

- LIME (Local Interpretable Model-Agnostic Explanations)
- SHAP (Shapley Additive Explanations)

LIME

The core idea of LIME is that even complex models behave approximately linearly in a small region around a single prediction.

Example: An ANN classifies a tissue as tumour/normal based on characteristics extracted from a medical image. LIME identifies which image characteristics most influence the model's prediction.

LIME provides a local explanation for a specific prediction, rather than a global understanding of how the model behaves.

LIME - Algorithm

1. Generate perturbed samples close to the original input.
2. Evaluate the original model on these samples.
3. Fit a simple interpretable model locally using the perturbed inputs and their predictions.
 - Linear models or decision trees are usually used as surrogate models.
4. Samples closer to the original input receive higher weights.
5. Use the surrogate model to generate an explanation.

SHAP

The SHAP method explains a prediction by answering “How much did each feature contribute to pushing the model output away from the average prediction ?”

It is based on the calculation of Shapley values considering the features as *players* and prediction as *reward*



Lloyd Stowell Shapley
2012 Nobel laureate

Shapley Value – Example 1

- A and B collaborate in a competition and win 1000 \$
- If A participate in the competition alone, A wins 500 \$
- If B participate in the competition alone, B wins 200 \$
- How should they share the 1000 \$ prize ?

value of A alone

value of A in a team with B

$$\phi_A = \frac{500 + 800}{2} = 650$$

$$\phi_B = \frac{200 + 500}{2} = 350$$

Shapley Value – Example 2

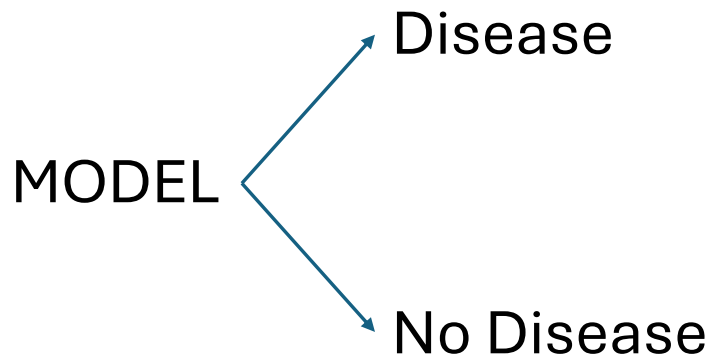
- A, B, and C collaborate in a competition and win 1000 \$
 - If A participate in the competition alone, A wins 500 \$
 - If B participate in the competition alone, B wins 200 \$
 - If C participate in the competition alone, C doesn't win anything
 - How should they split the 1000 \$ prize ? Should C get zero ?
-
- If A and B participate in the competition without C, they win 600 \$
 - If A and C participate in the competition without B, they win 800 \$
 - If B and C participate in the competition without A, they win 800 \$

- A, B, and C collaborate in a competition and win 1000 \$
- If A participate in the competition alone, A wins 500 \$
- If B participate in the competition alone, B wins 200 \$
- If A and B participate in the competition without C, they win 600 \$
- If A and C participate in the competition without B, they win 800 \$
- If B and C participate in the competition without A, they win 800 \$

	A	B	C
A→B→C	500	100	400
A→C→B	500	200	300
B→A→C	400	200	400
B→C→A	200	200	600
C→A→B	800	200	0
C→B→A	200	800	0
sum()/6	433	283	283

SHAP – For Feature Importance

- Features = Players
 - Prediction = Reward
- Compute how much each feature contribute to the final prediction

[illegible]

number of features in S

total number of features

Prediction of the model in x
using the feature set S

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{i\}) - f_x(S)]$$

All feature subsets that do not
include feature i

Prediction of the model in x
using feature set S plus the
feature i

Features: sex, age, bmi, ex. hours, cholesterol

i: bmi

S: sex, age $\rightarrow |S| = 2$

count the number of possible orderings for the remaining features (-1
because bmi is the 3rd one)

$$\frac{|S|! (|F| - |S| - 1)!}{|F|!} = \frac{2! (5 - 2 - 1)!}{5!} = \frac{1}{30}$$

it's a weighting factor that multiplies the contribution of
feature i across all possible feature subsets

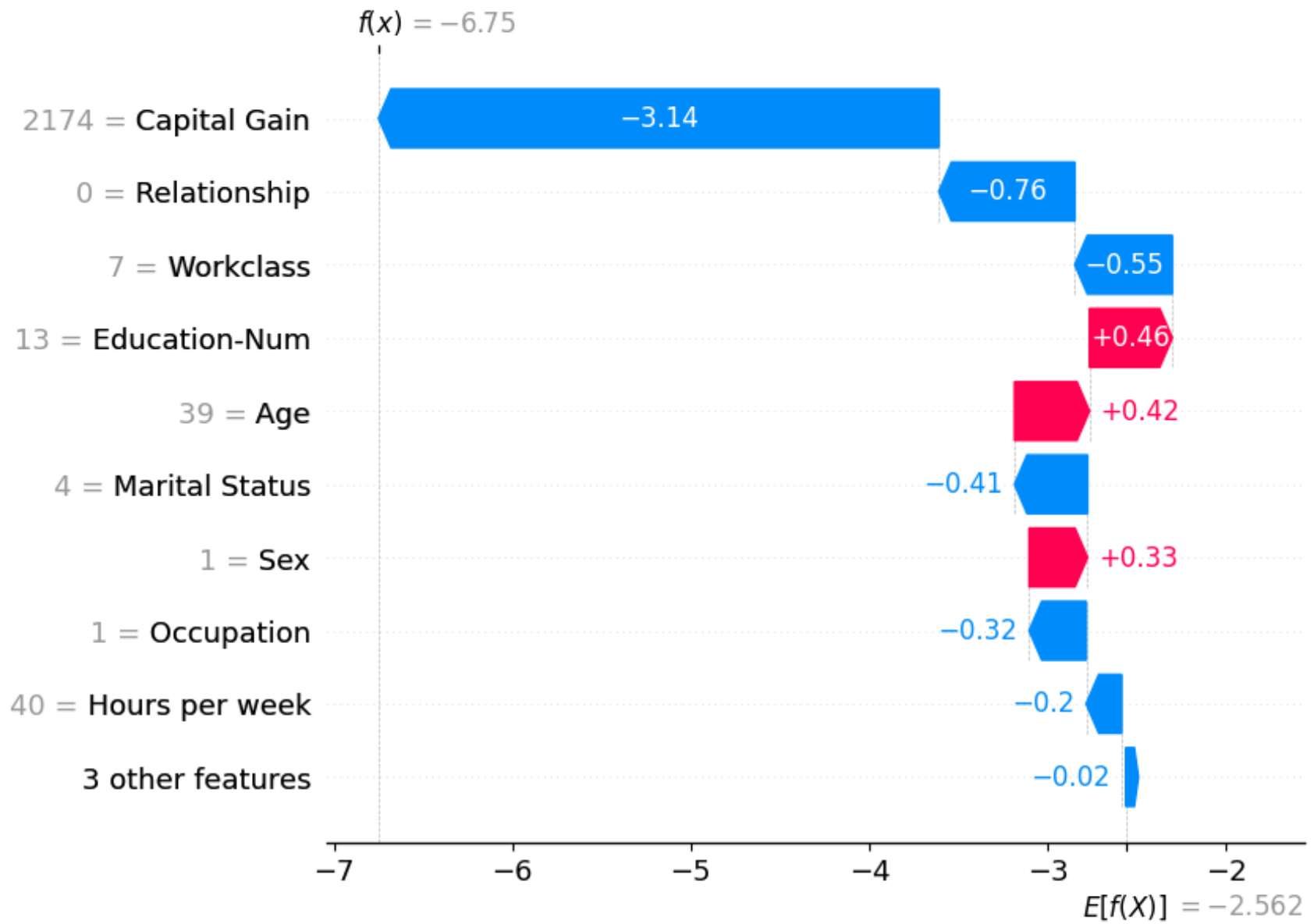
counts the number of possible orderings

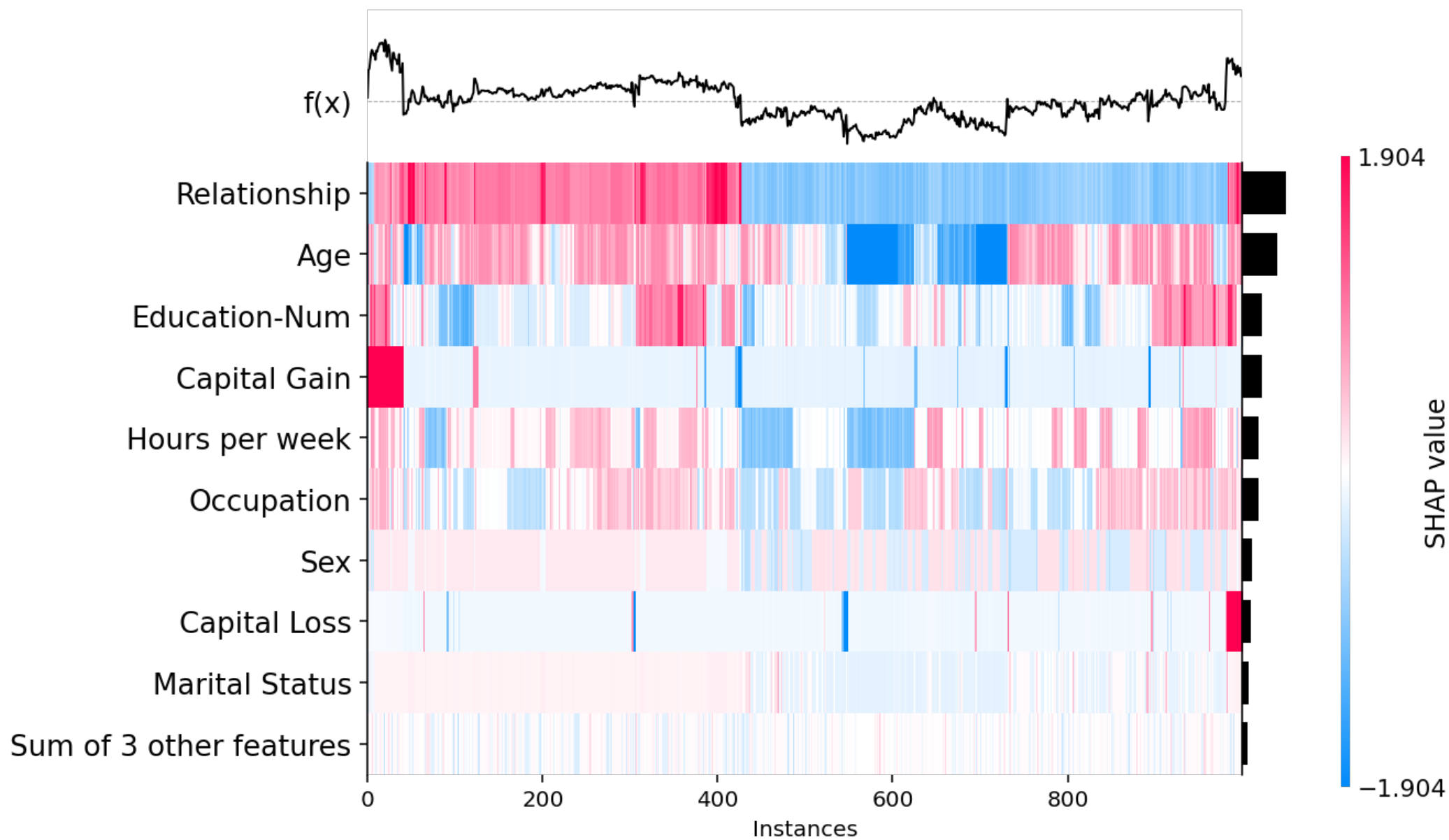
$$f(x) = E[f(x)] + \sum_i \phi_i(x)$$

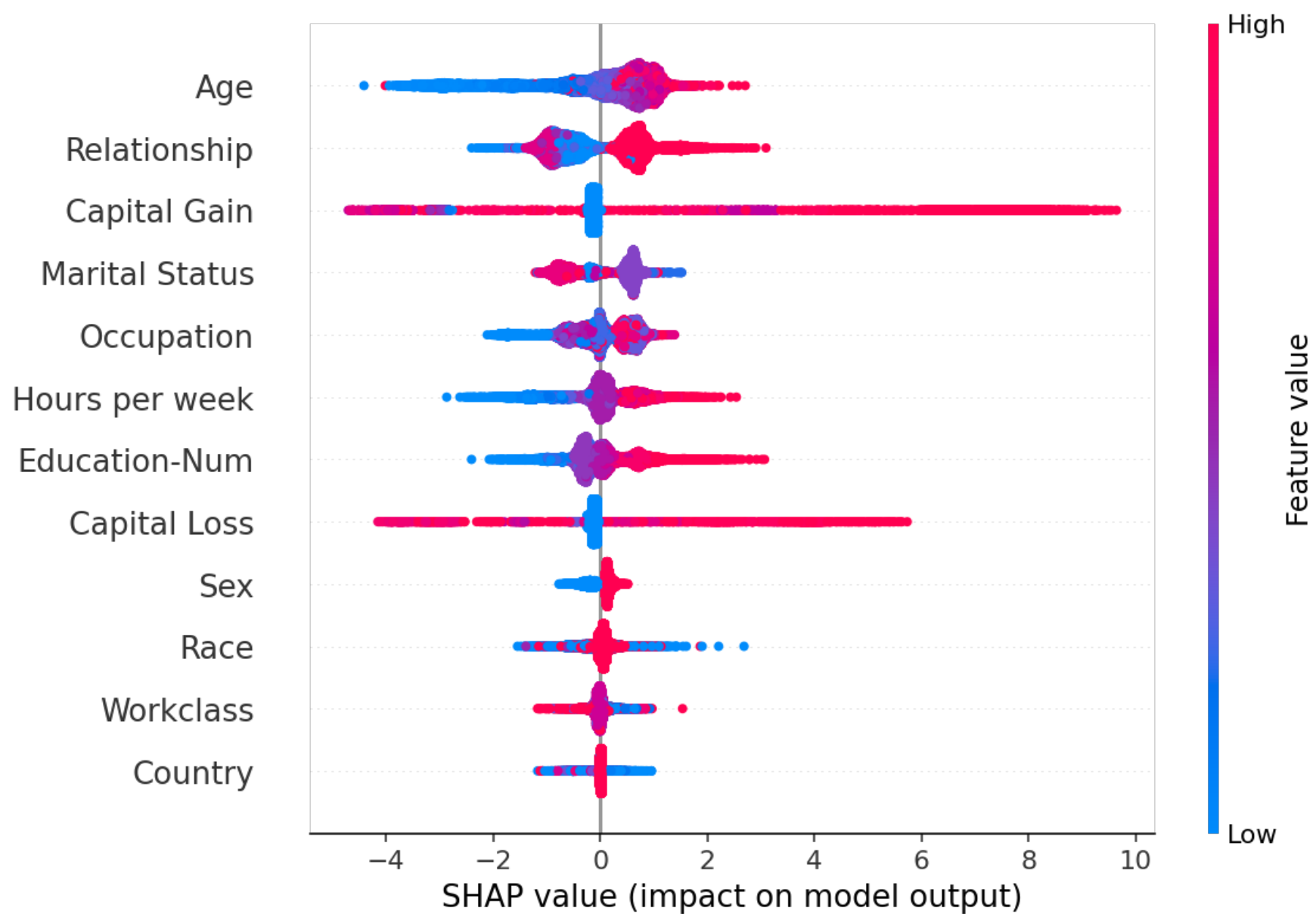
prediction of the model

average output

contribution of each feature to move away the prediction from the average

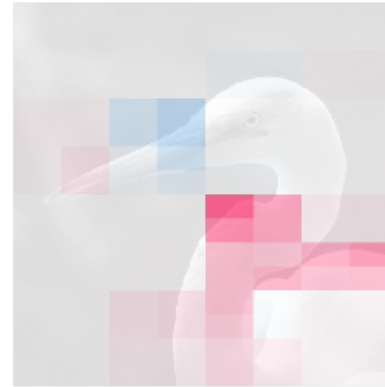




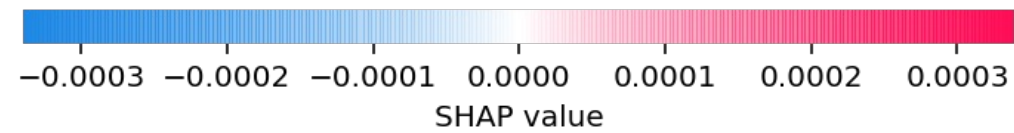
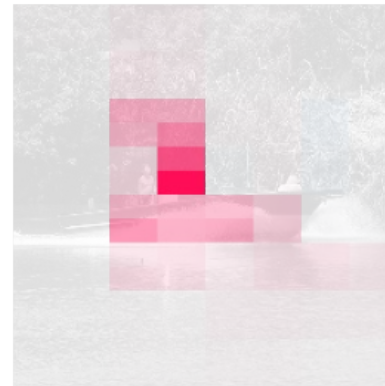




American_egret



speedboat

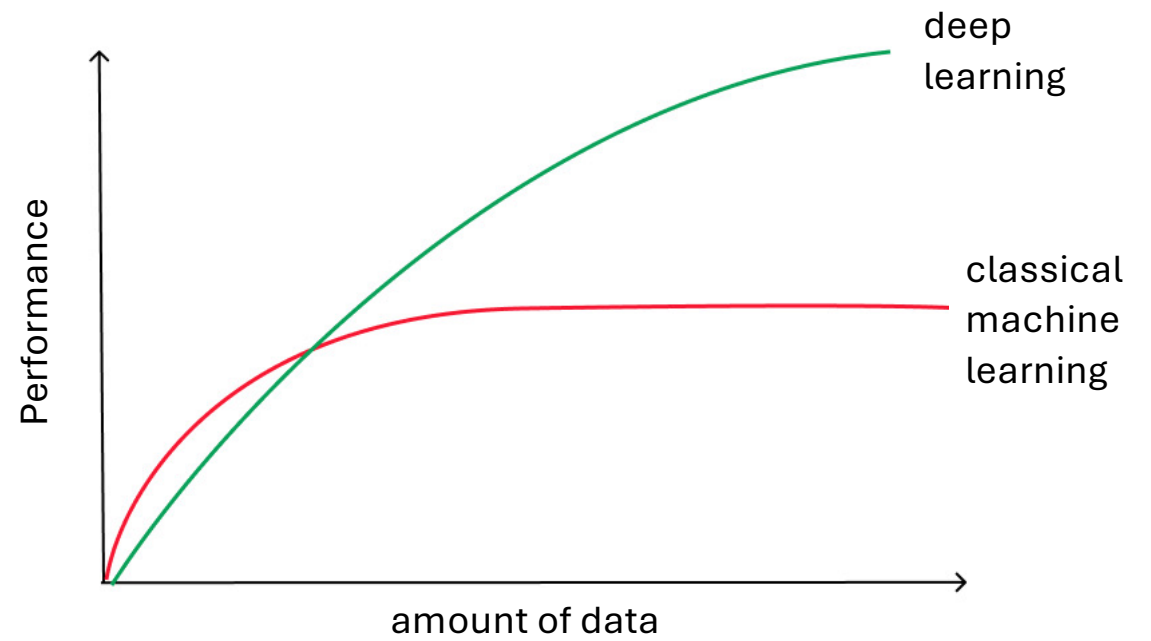
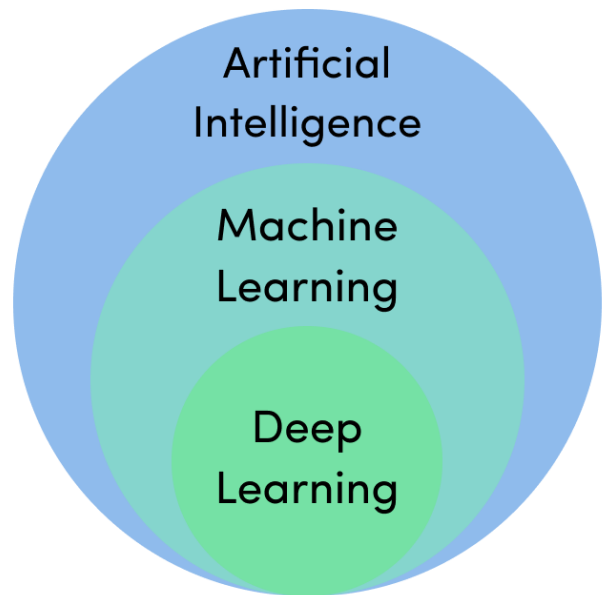


Limitations of SHAP

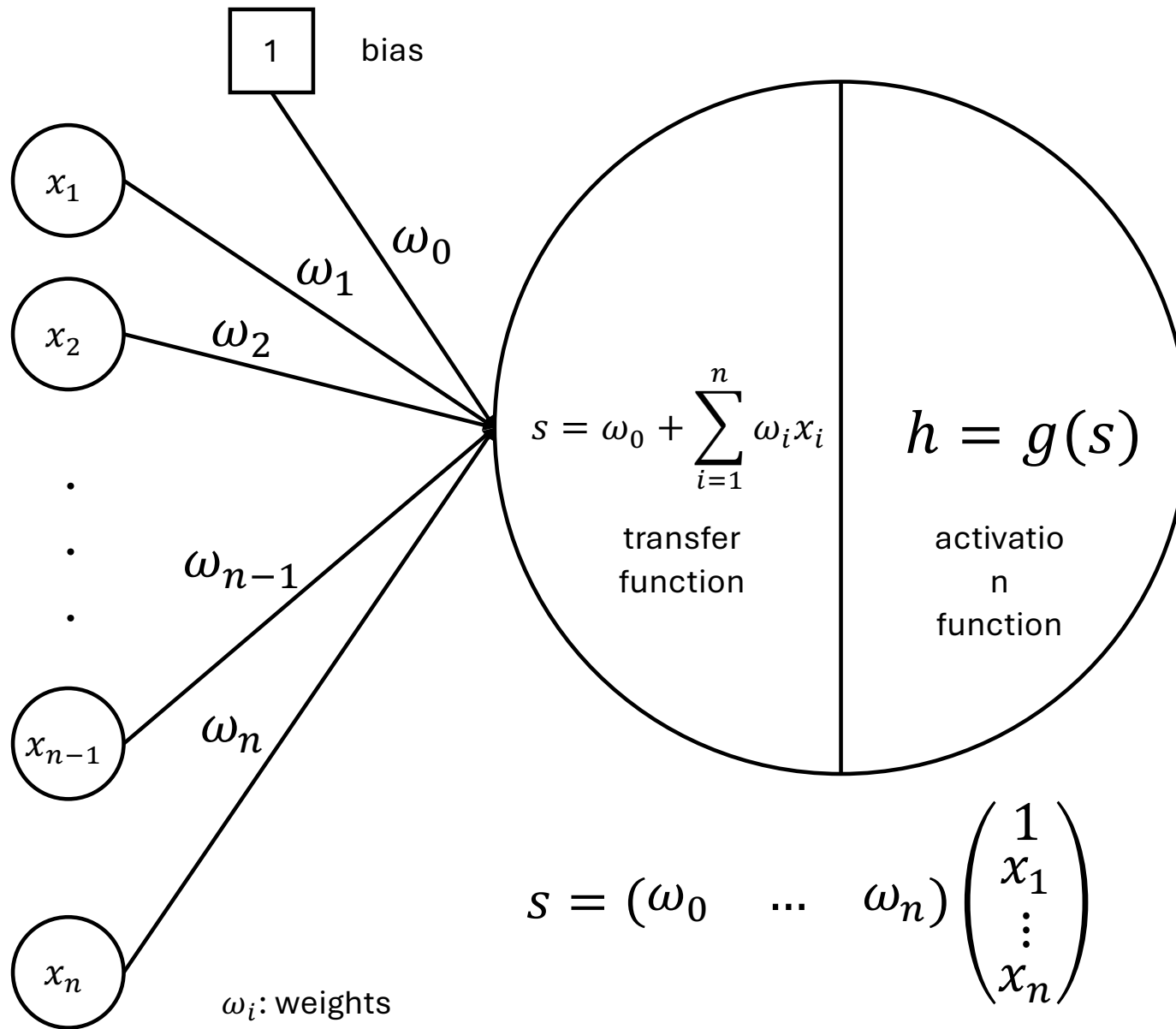
- Computational cost: SHAP values are expensive to compute
- Global interpretability: Combine SHAP values is complicated

- Clustering
 - KMeans
 - GMM
 - Hierarchical Clustering
 - DBSCAN
 - SOM
 - Elbow/Silhouette/AIC/BIC
 - Adjusted Rank Index
 - Why using metrics different than L2
- Dimensionality Reduction
 - PCA/kernel PCA
 - LDA
 - MDS/IsoMap
 - LLE
 - tSNE
 - UMAP
 - AE/VAE
 - KL divergence

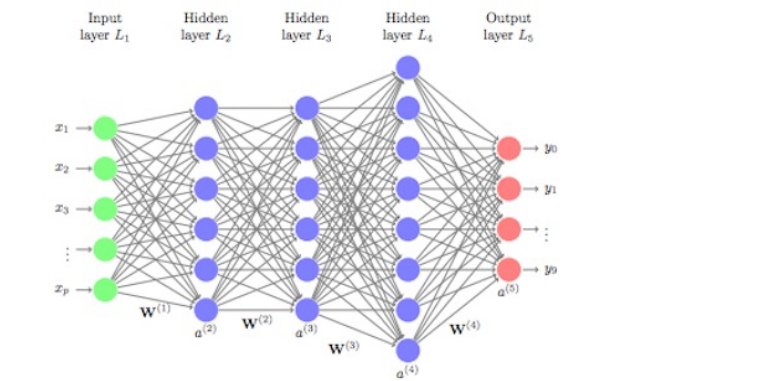
- Linear Models
 - Regression
 - Classification
 - Regularization strategies
 - L2, L1, elastic nets, Ridge
 - Fisher Information Matrix
 - Entropy and Cross-entropy
- Decision Trees
 - Measures of impurity
 - CART algorithm
 - Ensemble methods
 - Bagging
 - Random Forest
 - Adaptive Boosting
 - Gradient Boosting
- Support Vector Machines
 - Large margin classifiers
 - Kernels
 - Regularization
- Explainability
 - LIME
 - SHAP



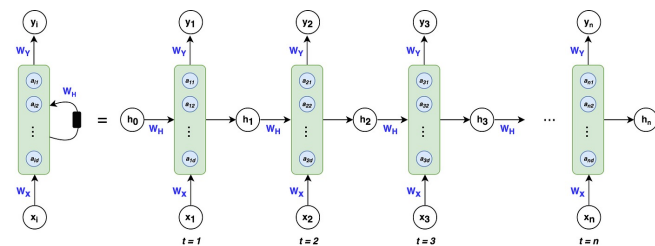
Artificial Neuron



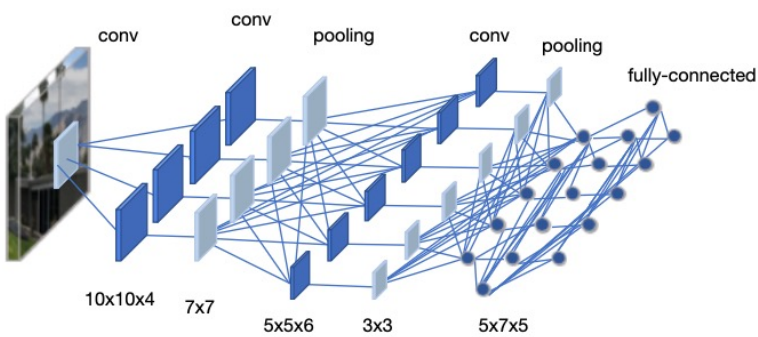
Feed Forward Neural Network



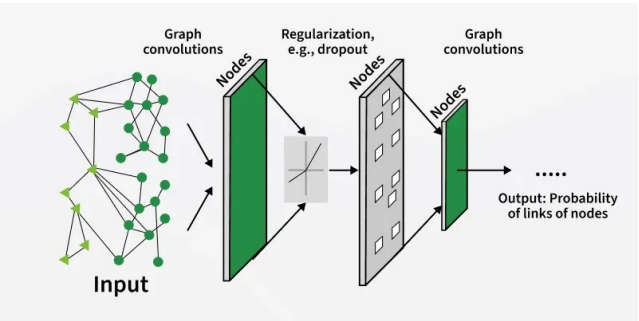
Recurrent Neural Network



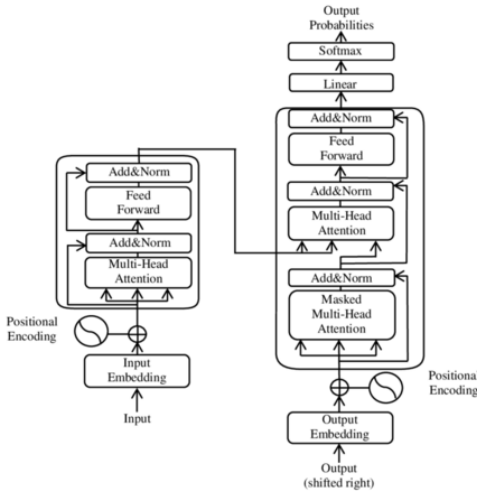
Convolutional Neural Network



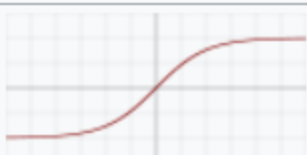
Graph Neural Network



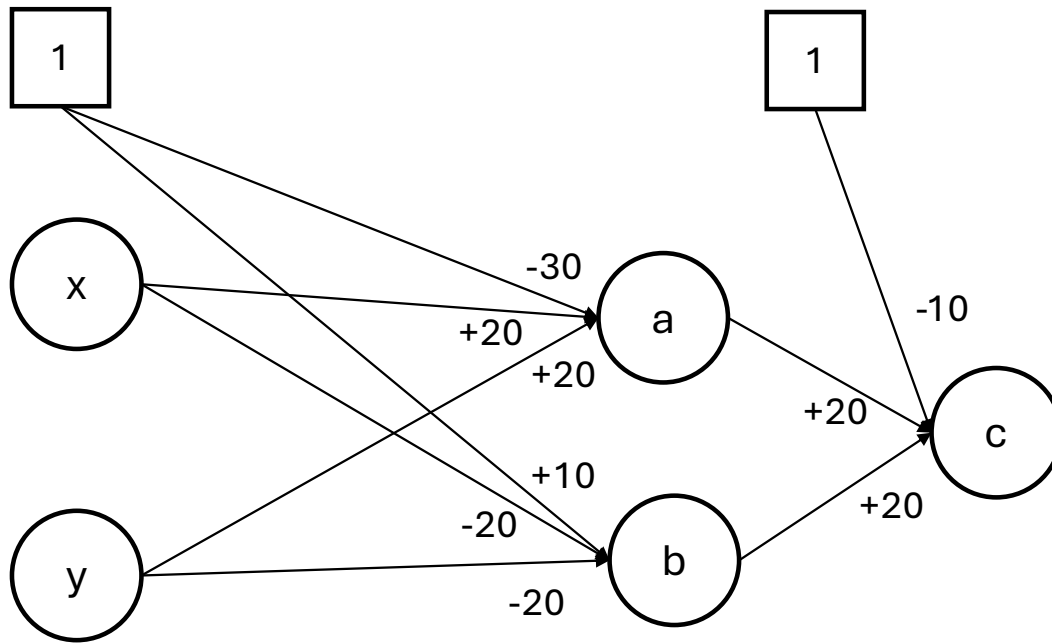
Transformer



- Neuron: the single unit of computation
 - Weights: strength of each connection
 - Transfer function: weighted sum of the neuron inputs
 - Activation function: function used to compute the output of the neuron
-
- Architecture: number and kind of neurons and how they are connected
 - Input layer, hidden layer, output layer
 - Loss/cost function: function used to train the network (estimate the weights)

Name	Plot	Function, $f(x)$
Identity		x
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$
Logistic, sigmoid, or soft step		$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic tangent (tanh)		$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Rectified linear unit (ReLU) ^[9]		$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max\{0, x\} = x \mathbf{1}_{x>0}$

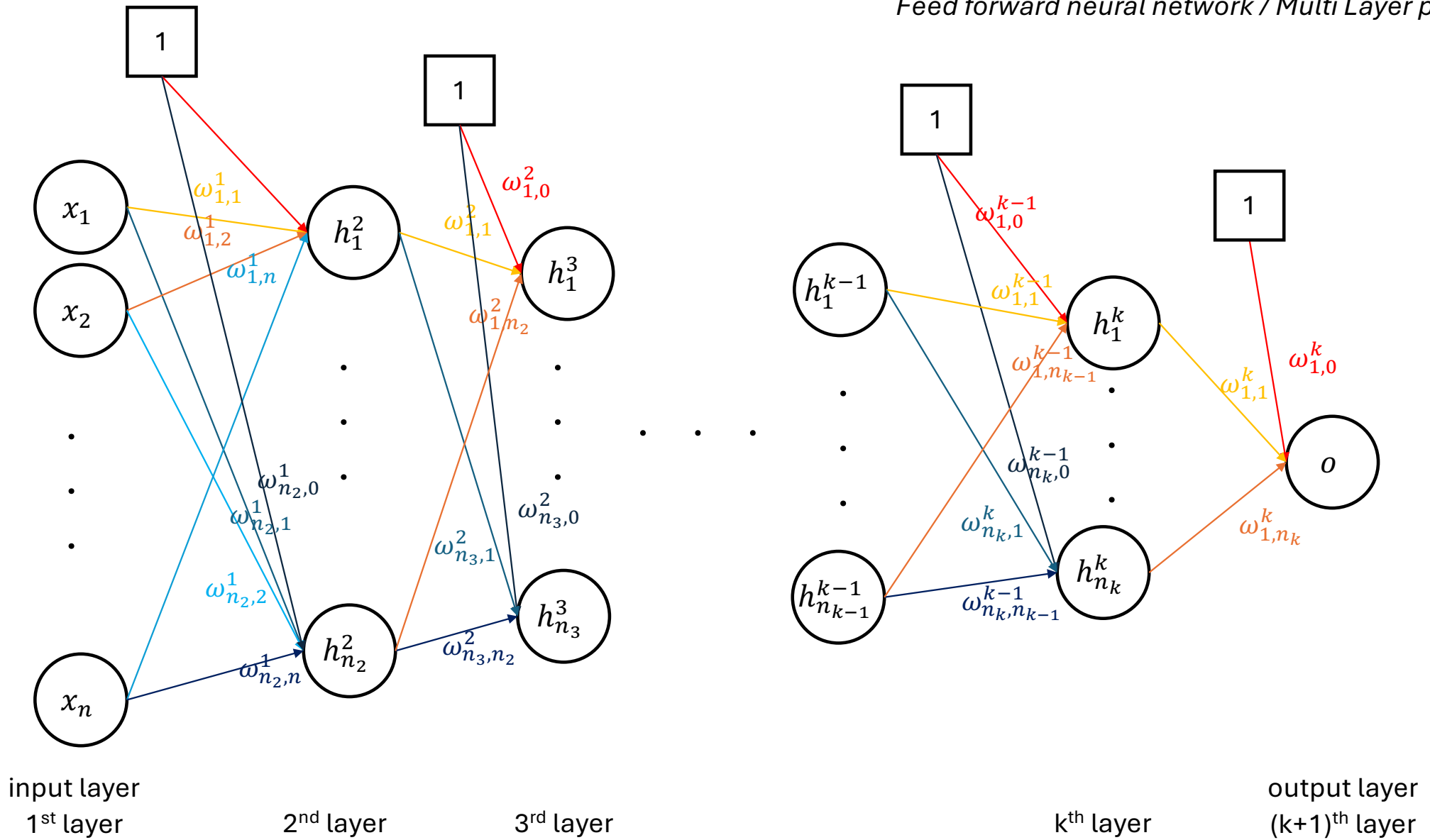
https://en.wikipedia.org/wiki/Activation_function

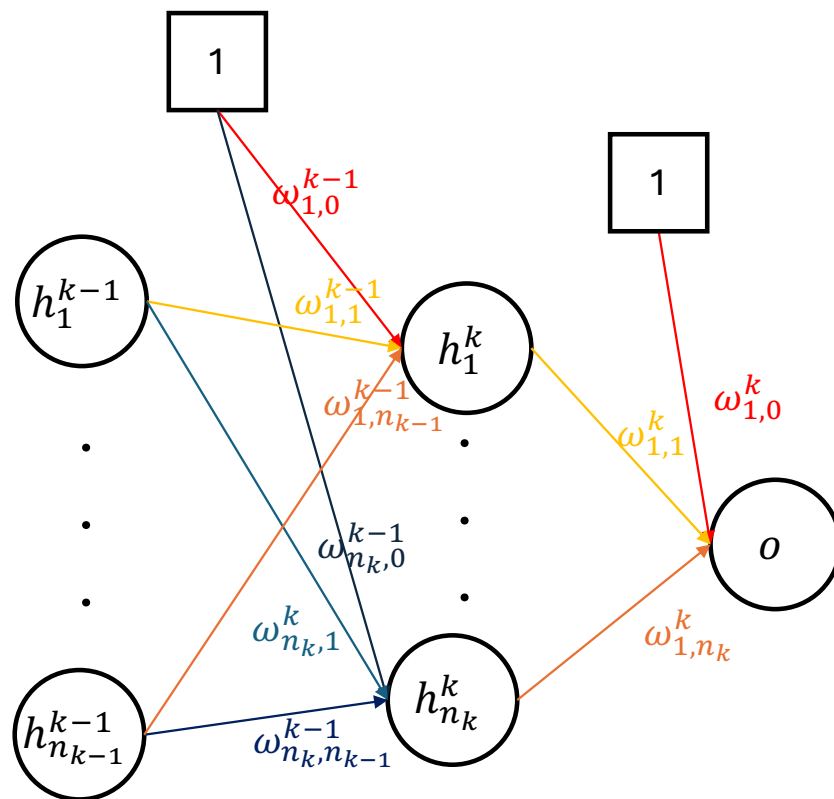
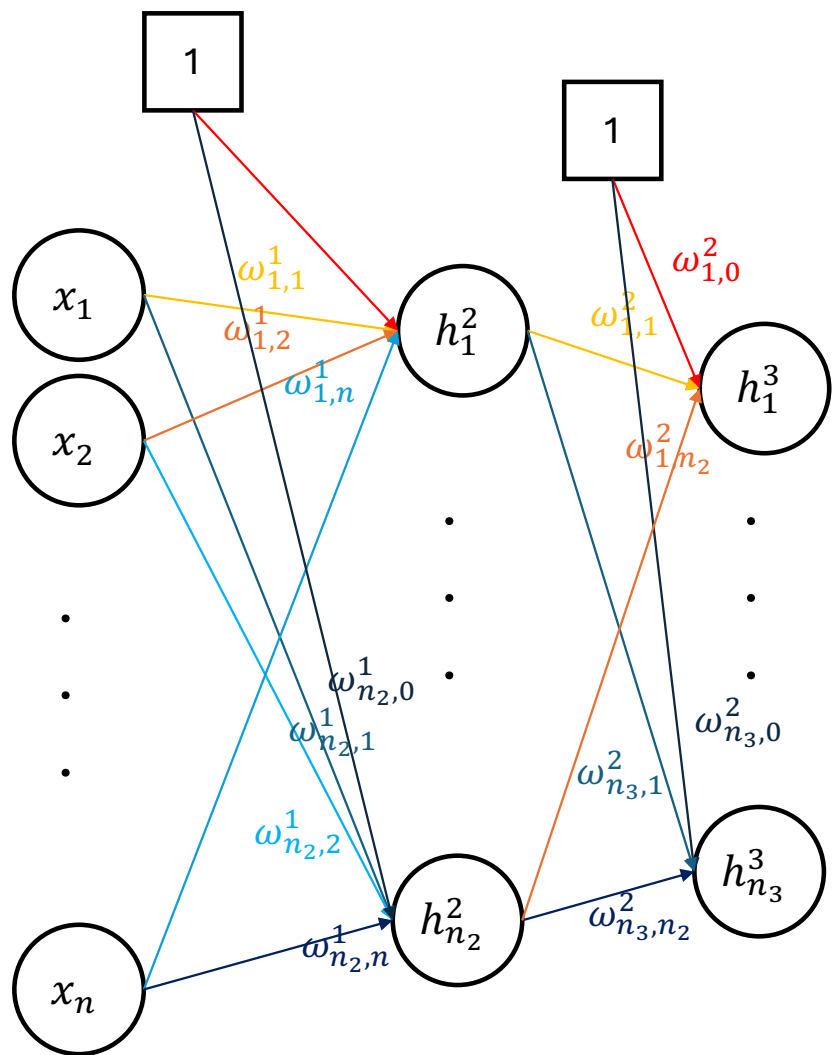


NOT(XOR)
with an ANN

x	y	a-input	a-output	b-input	b-output	c-input	c-output
0	0	-30	~0	+10	~1	+10	~1
0	1	-10	~0	-10	~0	-10	~0
1	0	-10	~0	-10	~0	-10	~0
1	1	+10	~1	-30	~0	+10	~1

Feed forward neural network / Multi Layer perceptron





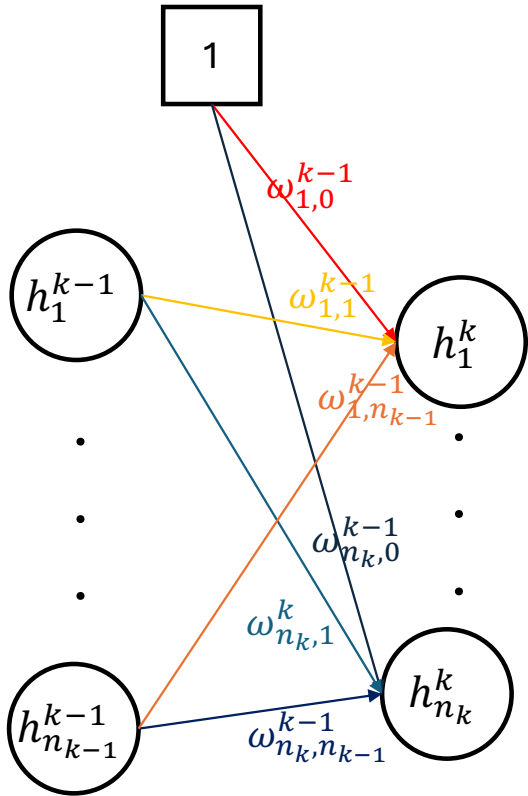
input layer
1st layer

2nd layer

3rd layer

k^{th} layer

output layer
 $(k+1)^{\text{th}}$ layer



$$\Omega^{k-1} = \begin{pmatrix} \omega_{1,0}^{k-1} & \omega_{1,1}^{k-1} & \dots & \omega_{1,n_{k-1}}^{k-1} \\ \omega_{2,0}^{k-1} & \dots & & \vdots \\ \vdots & \dots & & \vdots \\ \omega_{n_k,0}^{k-1} & \dots & & \omega_{n_k,n_{k-1}}^{k-1} \end{pmatrix}$$

$$\begin{pmatrix} \omega_{1,0}^{k-1} & \omega_{1,1}^{k-1} & \dots & \omega_{1,n_{k-1}}^{k-1} \\ \omega_{2,0}^{k-1} & \dots & & \vdots \\ \vdots & \dots & & \vdots \\ \omega_{n_k,0}^{k-1} & \dots & & \omega_{n_k,n_{k-1}}^{k-1} \end{pmatrix} \begin{pmatrix} 1 \\ h_1^{k-1} \\ \vdots \\ h_{n_{k-1}}^{k-1} \end{pmatrix} = \begin{pmatrix} s_1^k \\ s_2^k \\ \vdots \\ s_{n_k}^k \end{pmatrix}$$

$$\Omega^{k-1} \begin{pmatrix} 1 \\ h^{k-1} \end{pmatrix} = s^k$$

$$\begin{pmatrix} h_1^k \\ \vdots \\ h_{n_k}^k \end{pmatrix} = \begin{pmatrix} g_1^k(s_1^k) \\ \vdots \\ g_{n_k}^k(s_{n_k}^k) \end{pmatrix}$$

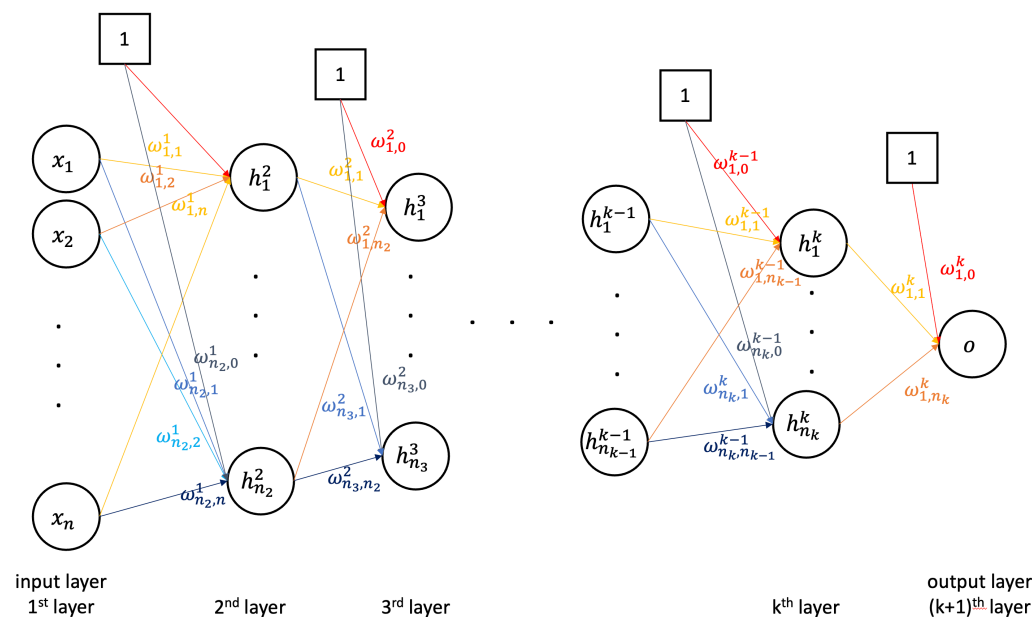
Forward Propagation

$$h^1 = x$$

for $j=2:k+1$

$$s^j = \Omega^{j-1} \begin{pmatrix} 1 \\ h^{j-1} \end{pmatrix}$$

$$h_j = g^j(s^j)$$



$$\Omega^{k-1} \begin{pmatrix} 1 \\ h^{k-1} \end{pmatrix} = s^k$$

$$\begin{pmatrix} h_1^k \\ \vdots \\ h_{n_k}^k \end{pmatrix} = \begin{pmatrix} g_1^k(s_1^k) \\ \vdots \\ g_{n_k}^k(s_{n_k}^k) \end{pmatrix}$$